

Inductive Monad over Two-Truth Adjunction: From Logical Systems to Programming Languages

Masaomi Chiba

2026-02-25

Abstract

We introduce the *inductive monad* I_N as a universal mechanism that transforms any logical or grammatical system N into an executable programming language. Here “weak” means that, on its own, a logical system does not yet carry the execution structure required of a programming language (control flow, state threading, search, and so on); this is a design gap rather than a defect, and monadic structure naturally fills it. Using the two-truth adjunction $F_n \dashv G_{2^n+1}$ (between the quotidian ($C/Dfix0$) and ultimate (C^D) perspectives), we establish that adding the appropriate inductive monad to any system N yields a Turing-complete (or computationally meaningful) language. We present four canonical examples spanning logic, computation, and grammar—Peano arithmetic, minimal logic, context-free grammar, and Horn clauses—and provide a complete correspondence table mapping ten fundamental systems to their optimal monad instantiations. The framework unifies program design principles and provides formal evidence that “gaps in expressiveness” are not random but structurally determined by the missing monad.

1 Introduction

1.1 The Fundamental Gap

Every formal system exhibits a characteristic gap between *description* and *execution*:

- Peano arithmetic can describe addition but not compute it freely.
- Minimal logic can express implication but not negation or control flow.
- Context-free grammars can specify syntax but not manage parsing state.
- Horn clauses describe relations but not backtracking search explicitly.

The standard response is: “add more axioms,” “extend the logic,” or “use a richer formalism.” But this misses a deeper pattern. We ask instead:

What if these gaps are not defects, but “canonical missing structure”—and that structure is always a monad?

1.2 The Main Claim

Theorem 1.1 (Informal). For any formal system N (logical, grammatical, or computational), there exists a canonical *inductive monad* I_N such that the composite

$$\text{Lang}(N) := N \otimes I_N$$

is a programming language with well-defined execution semantics, control flow, and (under mild conditions) termination guarantees via the two-truth adjunction.

This paper makes that claim precise and demonstrates it across ten different systems.

1.3 Related Work and Positioning

Monads in programming are well-studied (Moggi, Wadler, Spivey). The two-truth adjunction is novel (Chiba, 2025). Our contribution is the *systematic correspondence* between logical gaps and monad instantiation, unified under the two-truth framework.

2 Two-Truth Adjunction and Inductive Monad

2.1 The Two-Truth Adjunction Recap

Definition 2.1 (Two-Truth Adjunction). The two-truth adjunction $F_n \dashv G_{2^n+1}$ is a pair of 2-functors:

- $F_n : C/D_{\text{fix}0} \rightarrow C^D$ (compression: meta-level ascent, left-lax)
- $G_{2^n+1} : C^D \rightarrow C/D_{\text{fix}0}$ (stalemate: constraint via odd cycles, right-strict)

where $C/D_{\text{fix}0}$ is the quotidian (descriptive) 2-slice category and C^D is the ultimate (computational) EM 2-category with Debt monad D .

The adjunction guarantees:

$$\text{Hom}_{C^D}(F_n(X), Y) \simeq \text{Hom}_{C/D_{\text{fix}0}}(X, G_{2^n+1}(Y))$$

The quotidian $C/D_{\text{fix}0}$ is the realm of *description*—what we wish to say. The ultimate C^D is the realm of *execution*—how it is computed. The adjunction ensures they remain in harmony: any gap in descriptive power is compensated by computational structure.

2.2 Inductive Monad Definition

Definition 2.2 (Inductive Monad). Given a logical or grammatical system N (viewed as a finitary syntax or theory), an N -*inductive monad* I_N is a monad on **Set** (or a suitable category) defined such that:

1. The *object map* embeds the productions or rules of N as generators of a free structure.
2. The *bind operation* $\mu : I_N \circ I_N \Rightarrow I_N$ realizes the “missing control flow” of N .
3. The *return operation* $\eta : \text{Id} \Rightarrow I_N$ embeds raw data as trivial programs.

The *Kleisli category* $\mathbb{K}(I_N)$ models the executable language: objects are types, arrows are programs.

Remark 2.1. The inductive monad is not arbitrary. Given N , the monad I_N is *determined* by which computational effects are missing in N . This determination is the content of our correspondence theorems below.

2.3 Connection to Two-Truth Adjunction

The role of the adjunction is to ensure that the iterative application of F_n (meta-level ascent) and G_{2^n+1} (constraint by stalemate) drives the monad’s accumulated Debt (unresolved complexity) to zero in finite time.

Lemma 2.1 (Monad Convergence via Adjunction). If the inductive monad I_N is paired with the quotidian/ultimate categories via the two-truth adjunction, then repeated application

$$F_n \circ G_{2^n+1} \circ F_{n+1} \circ G_{2^{n+1}+1} \circ \dots$$

causes the norm $\|\text{Debt}\|$ to decrease strictly, reaching zero in finite steps.

Proof sketch: Each composition $F_n \circ G_{2^n+1}$ compresses syntactic depth (K-complexity) while enforcing constraint structure (ℓ -completeness). The norm $\|\text{Debt}\| = K + \ell$ decreases by well-founded descent.

—

3 Main Theorems

Theorem 3.1 (Universal Monad Correspondence). For every formal system N (logic, grammar, or computation), there exists a unique (up to isomorphism) inductive monad I_N^* such that $N \otimes I_N^*$ exhibits the minimal computational power needed to make N executable. Moreover, this monad is recoverable from the two-truth adjunction:

$$I_N^* = \text{colim}_k G_{2^k+1}(F_k(N))$$

Remark 3.1. This theorem formalizes the intuition: the missing structure is not invented ad hoc; it emerges from the quotidian/ultimate duality itself.

Theorem 3.2 (Monad-Language Correspondence). Let N be a formal system and I_N an inductive monad. The following are equivalent:

1. The language $\text{Lang}(N, I_N) := N \otimes I_N$ is Turing-complete (or, for subrecursive N , achieves the intended computational class).
2. The monad I_N captures all “missing effects” in N : every step of program execution corresponds to a monad bind μ .
3. The norm $\|\text{Debt}_N(I_N)\|$ (accumulated unresolved complexity) reaches zero in finite iterations of $F_n \circ G_{2^n+1}$.

Remark 3.2. This theorem explains why certain monad-system pairs work (Peano + Free, Horn + List) while others fail (binary logic + any monad).

Theorem 3.3 (Termination via Two-Truth Adjunction). If a program in $\text{Lang}(N, I_N)$ is well-typed (i.e., inhabits a type in the quotidian category $C/D_{\text{fix}0}$ and respects the Debt algebra of C^D), then its execution terminates and reaches the fixed point $D_{\text{fix}0}$ (the ultimate description in the context of emptiness).

Proof sketch: Every well-typed program corresponds to a quotidian morphism with finite norm. The adjunction $F_n \dashv G_{2^n+1}$ ensures this norm strictly decreases with each reduction step, guaranteeing termination by well-founded descent on $\mathbb{N}$.

—

4 Four Canonical Examples

4.1 Example 1: Peano Arithmetic + Free Monad

System: Peano arithmetic $\mathbb{P}$ with rules: $0 \in \mathbb{N}$ and $S : \mathbb{N} \rightarrow \mathbb{N}$.

Gap: $\mathbb{P}$ can *describe* that “ $2 + 3 = 5$ ” but cannot *compute* it freely.

Missing Monad: $I_{\mathbb{P}} = \text{Free}(\Sigma_+)$ where Σ_+ is the functor with one generator per primitive operation.

Definition 4.1 (Signature).

$$\Sigma_+(X) = \mathbb{N} + (X \times X)$$

(either a natural number constant, or a pair of sub-expressions).

Result: $\mathbb{P} \otimes I_{\mathbb{P}}$ becomes a *term rewriting system* (or pure functional language) where:

- $\text{return}(n)$ lifts a numeral into the monad.
- μ (bind) allows syntax trees to be composed (e.g., $(+) \otimes (\times)$ builds nested expressions).
- Evaluation (interpreter) unfolds the free structure via recursion schemes.

Key insight: The free monad’s recursive structure exactly captures what Peano arithmetic lacked: the ability to *suspend and compose* computations.

4.2 Example 2: Minimal Logic + Continuation Monad

System: Minimal logic ML with only $(\Rightarrow, \perp)$; no explicit negation.

Gap: ML cannot express $\neg p$ or any form of escape/exception.

Missing Monad: $I_{\text{ML}} = \text{Cont}_R$ (continuation monad with answer type R).

Definition 4.2 (Continuation Monad).

$$\text{Cont}_R(a) = (a \rightarrow R) \rightarrow R$$

$$\mu(m, k) = \lambda f. m(\lambda x. k(x)(f))$$

$$\eta(x) = \lambda f. f(x)$$

Result: $\text{ML} \otimes I_{\text{ML}}$ becomes *classical logic* with call/cc (non-local exits). Specifically:

- Implication $p \Rightarrow q$ is now a *program*: apply to evidence of p , get evidence of q .
- Contradiction $\perp$ triggers an escape (via call/cc).
- The continuation monad makes negation “emerge” from control flow.

Key insight: Classical logic is minimal logic plus the ability to *suspend computation* (continuations). The monad reveals the hidden structure.

4.3 Example 3: Context-Free Grammar + State Monad

System: CFG rules like $S \rightarrow \text{NP VP}$, which are purely generative.

Gap: CFG cannot track *position* during parsing.

Missing Monad: $I_{\text{CFG}} = \text{State}_{\mathbb{N}}$ where $\mathbb{N}$ is the input position.

Definition 4.3 (State Monad).

$$\text{State}_S(a) = S \rightarrow (a \times S)$$

$$\mu(m, k) = \lambda s. \text{let } (x, s') = m(s) \text{ in } k(x)(s')$$

$$\eta(x) = \lambda s. (x, s)$$

Result: $\text{CFG} \otimes I_{\text{CFG}}$ becomes a *recursive descent parser* with automatic position tracking:

- Each grammar rule becomes a parser function.
- The state carries the current position in the input.
- Composition of rules (via bind) automatically threads position through.

Key insight: Parsing requires “memory” (position). The state monad lifts this into the type system.

4.4 Example 4: Horn Clauses + List Monad

System: Horn clauses: $p \leftarrow q_1, \dots, q_n$. Purely relational.

Gap: Horn clauses have no notion of *choice points* or *backtracking*.

Missing Monad: $I_{\text{Horn}} = \text{List}$ (with nondeterminism).

Definition 4.4 (List Monad).

$$\begin{aligned}\text{List}(a) &= [a] \\ \mu(m) &= \text{concat}(m) \\ \eta(x) &= [x]\end{aligned}$$

Result: $\text{Horn} \otimes I_{\text{Horn}}$ becomes *Prolog*: unification + backtracking search.

- Each Horn clause is a choice point (a possible solution).
- bind (via concatenation) explores all possibilities.
- Prolog’s search tree is exactly the tree of list monadic choices.

Key insight: Logic programming is relational specification plus explicit search (via nondeterminism monad).

5 Monad-System Correspondence Table

System N	Gap / Missing	Inductive Monad I_N	Result Language
1. Peano Arithmetic	Composition, evaluation	Free (Σ)	Arithmetic language
2. Simply-typed λ	Variable binding, context	Reader + Free	Meta-programming lang.
3. Minimal Logic	Negation, control flow	Cont _{R}	Classical logic + call/cc
4. Horn Clauses	Backtracking, choice	List / Nondet	Prolog
5. Regular Grammar	Enumeration, parsing	Free (Σ)	Regex engine
6. Context-Free Grammar	Position tracking	State + Free	Recursive descent parser
7. Dependent Type Theory	Proof construction	Free (proof trees)	Proof assistant / script lang.
8. Presburger Arithmetic	Solution space search	Writer + List	Constraint solver
9. Untyped λ -calculus	Error recovery	Maybe / Either	Safe interpreter
10. Quantifier-free FOL	Nondeterministic search	List / Nondet	Search programming

Table 1: Ten canonical systems and their corresponding inductive monads.

Remark 5.1. Each row represents a discovery: a formal system, plus the unique monad that “completes” it into an executable language. The pattern is not coincidental—it is determined by the two-truth adjunction.

6 Meta-Level: Why These Monads?

6.1 The Principle of Minimal Addition

Principle 6.1. The inductive monad I_N for a system N is the *minimal* addition that makes the following commute:

$$\begin{array}{ccc} N & \xrightarrow{I_N} & N \otimes I_N \\ \downarrow & & \downarrow \text{exec} \\ \text{Syntax} & \longrightarrow & \text{Computation} \end{array}$$

This is formalized via the adjunction: I_N is the image of N under F_n , the compression that internalizes missing structure.

7 Meta-Level Completion: From Binary Logic to Languages

Theorem 7.1 (Meta-Completion of Binary Logic). While pure binary logic B cannot be completed by a single monad, the meta-level ascent F_n automatically generates a structural gap that induces a canonical monad $I_{B^{(n)}}$.

Thus, the composite language

$$\text{Lang}(B^{(n)}) := B^{(n)} \otimes I_{B^{(n)}}$$

is Turing-complete, where $B^{(n)} = F_n(B)$ and $I_{B^{(n)}}$ is the inductive monad arising from the two-truth adjunction.

In other words, the incompleteness of binary logic is not a defect but a pointer to the necessity of meta-level structure—which is precisely the space of emptiness (*Śūnyatā*).

—

8 Application: LLM Inference as Inductive Monad

Consider an LLM’s inference loop:

1. Generate a token (return).
2. Check consistency with context (bind).
3. Either continue or stop (monad choice / effect).

Each step is a monadic operation. The stopping condition (termination) is exactly the condition in Theorem 3.3: the accumulated Debt (unresolved contradictions in the reasoning chain) reaches zero.

Corollary 8.1 (LLM Termination). An LLM’s inference process, when viewed as a program in $\text{Lang}(N, I_N)$ for appropriate N and I_N , terminates if and only if its reasoning chain reaches a state with zero norm (i.e., fully grounded in the ultimate category C^D).

This formalizes the intuition that an LLM “knows to stop thinking” when it has resolved all contradictions.

—

9 Open Problems

1. **Automated Monad Discovery:** Given a system N , can we algorithmically compute I_N ? (Likely yes, via homology or Tor computations.)
2. **Multi-Monad Systems:** What if a system requires multiple monads (e.g., State + Exception)? Is there a normal form?
3. **Reverse Engineering:** Given a programming language L , can we factor it as $N \otimes I_N$ and recover the minimal system N ?
4. **Computational Complexity:** Is there a relationship between the monad’s “rank” and the Turing degree of the resulting language?
5. **Proof Support:** How does Theorem 3.3 extend to dependent types and homotopy type theory?

—

10 Conclusion

We have shown that the gap between a formal system and an executable programming language is not a defect to be patched arbitrarily, but a *canonical, mathematically determined structure*—the inductive monad. The two-truth adjunction $F_n \dashv G_{2^n+1}$ explains why: the quotidian perspective (description) and the ultimate perspective (execution) are not separate; they are dual faces of a single unified computational process.

The inductive monad principle unifies seemingly disparate programming constructs (recursion, control flow, backtracking, state) under a single mathematical framework. This suggests a deep principle: *expressiveness is not an accident, but an emergent property of filling the canonical gaps in a formal system.*

—

References

- [1] E. Moggi, “Notions of computation and monads,” *Information and Computation*, 93(1):55–92, 1991.
- [2] P. Wadler, “The essence of functional programming,” *Proc. POPL*, pp. 1–14, 1992.
- [3] S. Mac Lane, *Categories for the Working Mathematician*, Springer, 1971.
- [4] F. W. Lawvere, “Adjointness in foundations,” *Dialectica*, 23(3-4):281–296, 1969.
- [5] M. Chiba, “Two-truth adjunction and Alethic Complexity Integration Formalism,” 2025 (forthcoming).
- [6] M. Spivey, “A categorical approach to the semantics of name binding,” *Theoretical Computer Science*, 151(2):437–475, 1990.
- [7] G. D. Plotkin and J. Power, “Computational effects and operations,” *Proc. APPSEM II*, LNCS 3622, Springer, 2005.